Reporte Técnico de Actividades Práctico-Experimentales Nro. 05

1. Datos de identificación del estudiante y la práctica

Nombre del estudiante(s)	Maria Soledad Buri Camacho
Asignatura	Simulacion
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una simulación bajo principios de transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamän Q.
Fecha	29 de Octubre de 2026 30 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la práctica

Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.

- Implementar un generador LCG en Python.
- Implementar el mismo algoritmo en Java.

- Comparar las secuencias generadas y analizar sus propiedades

3. Materiales, reactivos, equipos y herramientas

- Python 3.
- Java/JDK

4. Procedimiento / Metodología Ejecutada

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, aplicadas a la implementación y comparación del Generador Congruencial Lineal (LCG) en Python y Java.

Etapa 1. Definición del problema

- Se planteó la necesidad de generar números pseudoaleatorios mediante el método LCG, con el objetivo de comparar las secuencias obtenidas en Python y Java.

Etapa 2. Formulación conceptual

- Se definió el funcionamiento del generador LCG mediante la fórmula $X(n+1) = (a \cdot X(n) + c) \bmod m$ y la normalización $U(n) = X(n)/m$.

Etapa 3. Recolección y análisis de datos

- Se utilizaron los parámetros establecidos en la guía: a = 1664525, c = 1013904223, $m = 2^{32}$ y semilla inicial = 12345, para generar 10 000 valores.

Etapa 4. Construcción del modelo

- Se implementó el algoritmo LCG en Python y Java, utilizando la misma fórmula y los mismos parámetros para generar las secuencias pseudoaleatorias.

Etapa 5. Verificación y validación

- Se comprobó la correcta ejecución de ambos programas y la coincidencia de los 10 000 valores generados, comparando especialmente las primeras 20 observaciones.

Etapa 6. Experimentación y análisis de resultados

- Se graficaron las secuencias, se analizó el período del generador y se cambió la semilla a 54321 para observar las diferencias entre los resultados.

Etapa 7. Implementación y documentación

- Se organizaron los programas, las tablas comparativas y las gráficas obtenidas. Finalmente, se documentaron los resultados y se elaboraron las conclusiones de la práctica.

5. Resultados

Resultados estadisticos

Indicador	Semilla 12345	Semilla 54321
Valores generados	10000	10000
Media de Un	0.498770	0.498870
Mínimo de Un	0.000059	0.000078
Máximo de Un	0.999926	0.999846
X1	87628868	1238253532
X2	71072467	1708513675

Al cambiar la semilla de 12345 a 54321, se obtuvieron secuencias pseudoaleatorias diferentes, aunque ambas generaron 10 000 valores. Las medias fueron cercanas a 0.5 y los valores normalizados se mantuvieron entre 0 y 1. Esto demuestra que la semilla influye en la secuencia generada, mientras que los parámetros del algoritmo LCG se mantienen constantes.

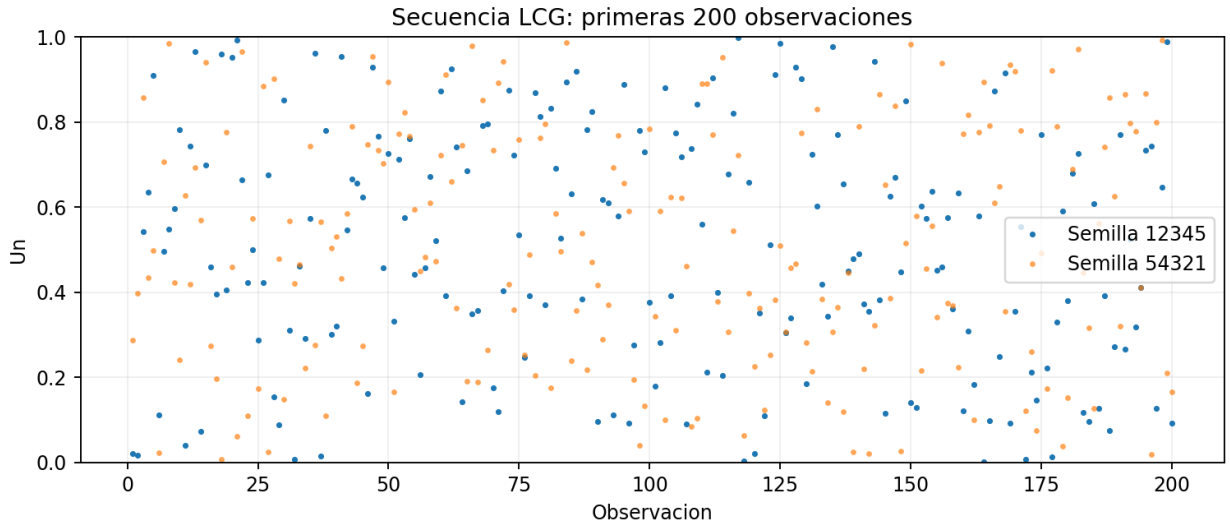


Figura 1. Primeras 200 observaciones para las dos semillas.

Los valores aparecen dispersos entre 0 y 1 y no muestran una tendencia creciente o decreciente evidente en este tramo. Las series no son iguales porque se utilizaron semillas diferentes; la gráfica por sí sola no demuestra aleatoriedad estadística.

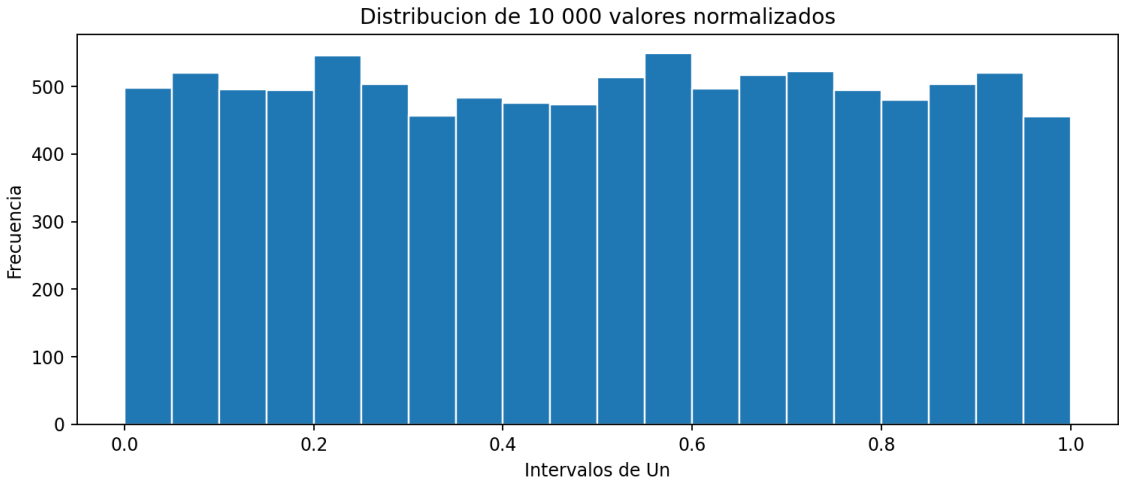


Figura 2. Histograma de las 10 000 observaciones con semilla 12345.

Las frecuencias de los intervalos son relativamente similares y la media (0,498770) es cercana a 0,5. Esto es una comprobación descriptiva, no una prueba estadística completa de uniformidad o independencia.

6. Preguntas de Control

1. ¿Qué significa pseudoaleatorio?

- Significa que una secuencia parece aleatoria, pero es calculada por reglas matemáticas y puede reproducirse.

2. ¿Qué función cumple la semilla?

- Es el valor inicial X0. Si se mantiene la misma semilla y los mismos parámetros, se genera exactamente la misma sucesión.

3. ¿Qué significa período?

- Es la longitud del ciclo de estados del generador antes de que se repitan.

4. ¿Qué ocurre si se utilizan parámetros inadecuados?

- Pueden aparecer ciclos cortos, patrones notorios o valores poco útiles para una simulación.

5. ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?

- Permite verificar los cálculos y comprobar que los resultados dependen del algoritmo, no del lenguaje empleado.

7. Conclusiones

- Se implementó el método LCG en Python y Java con los parámetros indicados en la guía y se obtuvieron 10 000 observaciones por ejecución.

- La comparación confirmó que los 10 000 valores calculados por ambos lenguajes coinciden. Los primeros 20 quedaron documentados en una tabla y en un CSV.
- Las gráficas presentan valores distribuidos a lo largo del intervalo [0, 1); este comportamiento visual no basta para certificar su calidad estadística.

8. Recomendaciones

- Ejecutar el generador LCG con diferentes semillas para analizar cómo cambian las secuencias de números pseudoaleatorios.
- Mantener los mismos parámetros del generador en Python y Java al realizar comparaciones, para comprobar que ambos programas producen los mismos resultados.

9. Reflexión Critica

- La práctica permitió comprender cómo funciona la generación de números pseudoaleatorios mediante el método LCG y cómo Python y Java pueden producir las mismas secuencias utilizando parámetros iguales. Asimismo, se reconoció la importancia de seleccionar adecuadamente los parámetros y la semilla inicial para obtener resultados reproducibles y comprender las limitaciones del generador.

10. Anexos

Link del GitHub:

https://github.com/Zoe777zz/Practica_05_Simulacion_5A.git

Declaración de uso de IA Generativa:

1. Herramienta de Inteligencia Artificial utilizada

- ChatGPT, desarrollada por OpenAI.

2. Forma en que se utilizó la herramienta

- Se utilizó ChatGPT como herramienta de apoyo para la implementación del Generador Congruencial Lineal (LCG) en Python y Java. Su uso se centró en la organización del código, la aplicación de la fórmula matemática, la comparación de las secuencias pseudoaleatorias y la generación de gráficas para el análisis de los resultados.

3. Prompts usados

- **Prompt 1:** Ayúdame a implementar el Generador Congruencial Lineal (LCG) en Python y Java, utilizando los parámetros establecidos. Organiza el código de forma sencilla, aplicando el principio KISS y utilizando nombres en español.